NAVIGATION DESIGN DOCUMENT V1.0

Obstacle Course Navigation Design

How we navigate the 2026 DIY Robot Challenge obstacle course — zone-aware state machine, map-free costmap strategy, dual EKF fusion, and sensor-loss recovery for the tunnel section. No static .pgm map required.

COMPETITION	AUG 14 DEMO	PLATFORM	TEAM
October 1, 2026	Autonomous Motion	Jetson Orin Nano	Juggernauts

ROS 2 Humble · FAST-LIO2 · NDT-OMP · Nav2 MPPI · Dual EKF · No PGM Map · Zone State Machine

Table of Contents

1 The Problem with a Static Map

The standard Nav2 setup uses a **static occupancy grid** — a .pgm image file — loaded by `map_server` to give the global planner a pre-known picture of the environment. This works well indoors on flat surfaces. For our outdoor obstacle course it breaks down in several ways.

Why PCD → PGM Fails Outdoors

LIO-SAM gives us a 3D point cloud (`GlobalMap.pcd`). Converting it to a 2D .pgm requires projecting down to a single horizontal slice. On our course:

- Ramps make the floor non-flat — a 2D slice shows the ramp as a wall
- Gravel gives noisy lidar returns — the free space ends up with gaps and holes in it
- Bucket positions are baked into the map at recording time — if they move before competition, the map is wrong
- Tunnel ceiling returns appear as impassable walls in 2D
- The map origin and orientation must match exactly — easy to get wrong

What We Do Instead

We skip the static layer entirely. Here's the insight:

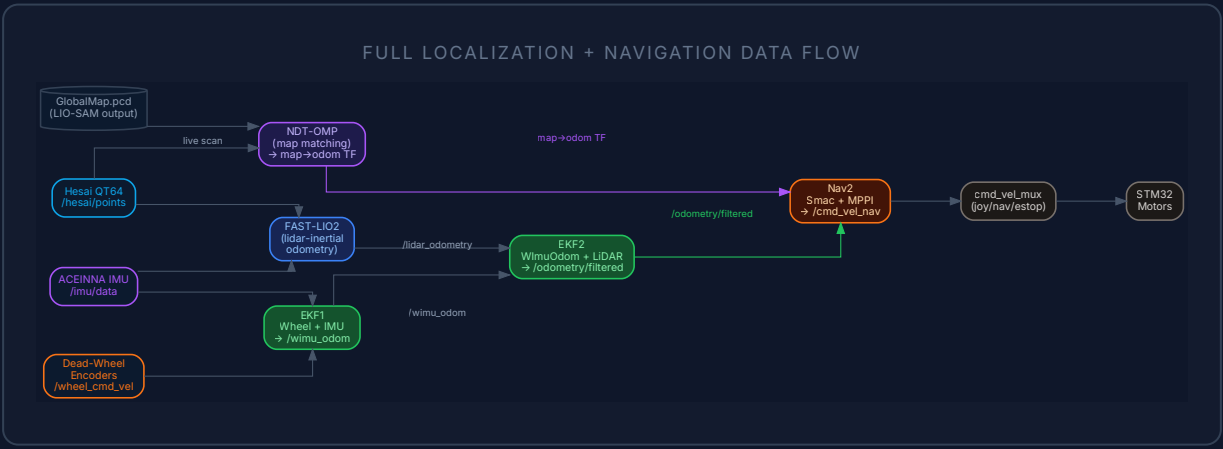
- **Localization** (where am I?) is solved by NDT-OMP matching live scans against `GlobalMap.pcd` directly — no 2D projection
- **Obstacle avoidance** (what's around me?) is solved by the live lidar feeding the `obstacle_layer` in real time
- **Path planning** works fine without a static layer using `allow_unknown: true`
- Dynamic obstacles — like randomly placed buckets — are always current

KEY INSIGHT

The .pgm file was only ever needed to tell the planner where the walls are. Our lidar sees those walls live at 10 Hz. We already have better, fresher information — we just need to use it correctly.

2 Localization Stack — No .pgm Needed

The full localization pipeline is shown below. It has three independent pieces that each answer a different question: NDT-OMP answers where am I globally, FAST-LIO2 answers how am I moving, and the dual EKF answers what is my best smoothed estimate.



2.1 LIO-SAM — Offline Map Generation (done once before competition)

We drive the course once with LIO-SAM running. It builds a globally consistent 3D point cloud map using a factor graph with loop closure. When we're happy with the map quality in RViz, we call the `/lio_sam/save_map` service and get three files: `CornerMap.pcd`, `SurfMap.pcd`, and most importantly `GlobalMap.pcd` — the combined map that everything else works from.

> **MAP QUALITY IS FOUNDATION**
>
> The quality of this single mapping run determines how well NDT-OMP localises on competition day. Drive slowly, complete full loop closure, verify in RViz before saving. We need to build this map early — it's the single most important pre-competition task.

2.2 NDT-OMP — Runtime Map Matching

NDT-OMP (Normal Distributions Transform with OpenMP parallelism) loads `GlobalMap.pcd` at startup and continuously matches each incoming Hesai scan against it. The output is the **map→odom transform** — this is what tells Nav2 where the robot is in the global frame. It publishes directly into the TF tree, so Nav2 doesn't need to know anything about how it works.

This replaces AMCL entirely. Unlike AMCL, NDT-OMP works in 3D and doesn't need a 2D map. It is already in our repo as the `ndt_omp_ros2` submodule.

2.3 Dual EKF Fusion

FAST-LIO2 runs at 10 Hz — once per lidar scan. Between scans there's a 100ms gap where the robot's pose estimate doesn't update. For the MPPI controller running at 20 Hz, this gap causes jerky velocity commands. The dual EKF eliminates this.

EKF1 — High Rate Local (ekf_wimu.yaml)

Fuses dead-wheel encoder odometry (`/wheel_cmd_vel`) with IMU angular velocity (`/`

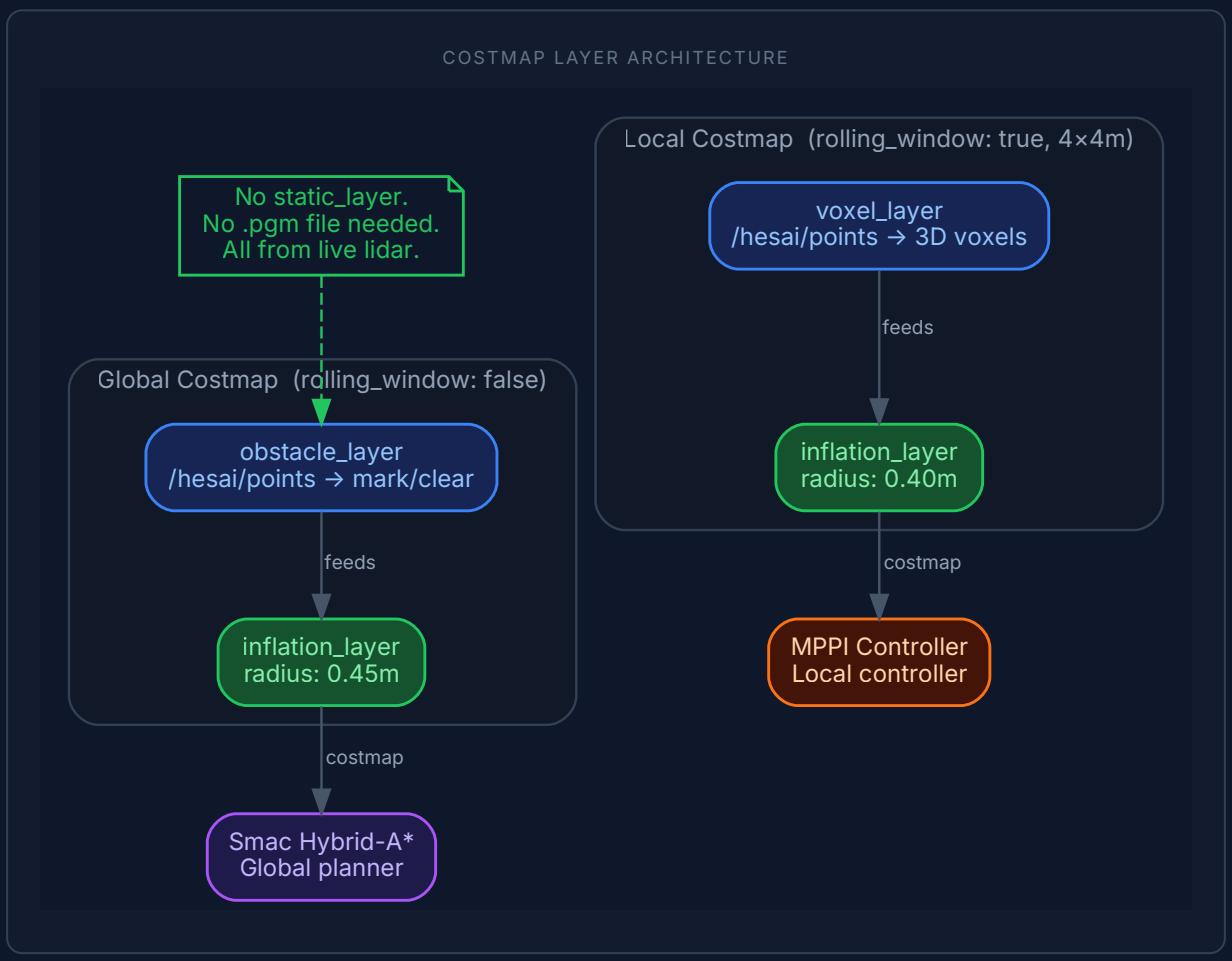
EKF2 — Accurate Fused Output (ekf_local.yaml)

`imu/data`) to produce `/wimu_odom` at 50–100 Hz. This is fast, smooth, and fills the gaps between lidar updates. It drifts over time but is excellent over short intervals.

Takes `/wimu_odom` (high-rate smooth) and `/lidar_odometry` (FAST-LIO2, accurate, lower rate) and fuses them into `/odometry/filtered`. The result is both smooth and accurate — MPPI gets 50 Hz smooth inputs, drift is corrected every lidar scan.

3 Costmap Strategy — Live Lidar Only

Nav2's costmap system has a modular plugin architecture. Each plugin contributes information to a shared 2D grid. We use **only the live lidar** — no static layer, no pre-built map. This is a deliberate design choice, not a compromise.



How the Global Costmap Builds Up Without a Static Layer

The global costmap has `rolling_window: false` — it doesn't clear itself as the robot moves. Every obstacle the Hesai lidar sees gets marked in the grid and **stays there**. As the robot drives around the course, the costmap accumulates a complete picture of what's been observed. By the time the robot needs to plan a long path, it has already seen most of the environment.

The planner is configured with `allow_unknown: true` — it will plan through unseen space if needed. On a fixed known course, this is safe because the course boundaries are quickly observed and marked.

obstacle_layer

Subscribes to `/hesai/points`. Marks cells occupied when a point lands within `max_obstacle_height: 2.0m` and `obstacle_max_range: 5.0m`. Raycasts back to robot

inflation_layer

Inflates every occupied cell by `inflation_radius: 0.45m` with a cost gradient. The planner naturally routes paths away from walls. MPPI uses the cost gradient for smooth avoidance.

static_layer — removed

Not used. This was the only plugin that required a `.pgm` file. With it gone, `map_server` still runs (for the map→odom TF chain) but loads nothing. Three lines removed from `nav2_params.yaml`.

to clear free space. Updates at 10 Hz.

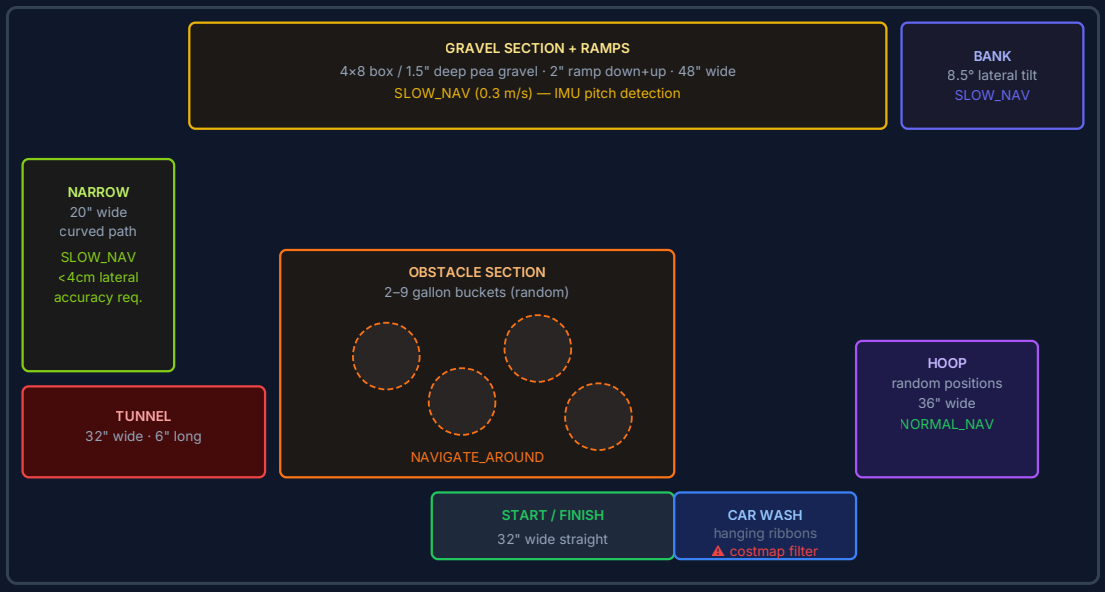
DYNAMIC OBSTACLES ARE BETTER THAN STATIC

The bucket obstacles in the obstacle section are randomly placed — they cannot be pre-mapped. The live obstacle_layer handles them perfectly. Buckets that moved between laps are automatically updated. This is actually better than a static map for this course.

4 Obstacle Course — Zone Analysis

The 2026 course is 65' × 48' and contains ten distinct sections, each posing a different challenge to the navigation stack. The map below shows each section colour-coded by the navigation mode it requires.

COURSE ZONE MAP — NAVIGATION MODES

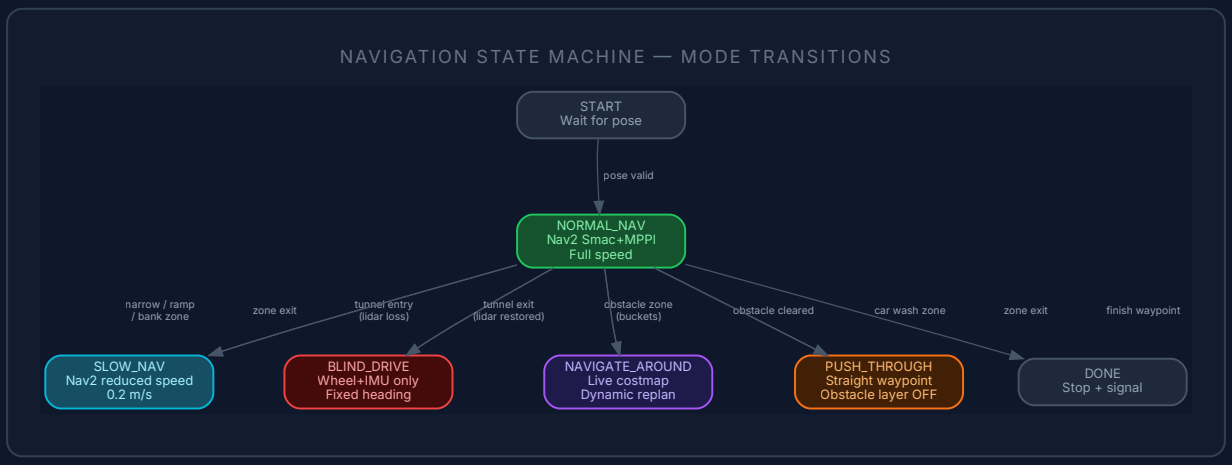


SECTION	KEY DIMENSIONS	PRIMARY CHALLENGE	FAST-LIO2 STATUS	NAVIGATION MODE
Start / Finish	32" wide, flat	Clean start, consistent positioning	✅ Full	NORMAL_NAV
Car Wash	48" wide	Hanging ribbons register as lidar wall	⚠️ Corrupted	PUSH_THROUGH
Hoop Section	36" wide	Random hoop positions on path	✅ Full	NORMAL_NAV
Obstacle (buckets)	~11' × 26' open	2–9 randomly placed 5-gallon buckets	✅ Full	NAVIGATE_AROUND
Gravel + Ramps	48" wide, 2" ramp	Surface noise, pitch change, wheel slip	⚠️ Degraded	SLOW_NAV
Bank Section	48" wide, 8.5° tilt	Lateral roll corrupts 2D odometry	⚠️ Minor	SLOW_NAV
Narrow Section	20" wide, curved	<2" clearance — needs <4cm lateral accuracy	✅ Full	SLOW_NAV
Tunnel	32" wide, 6' long	Foil-lined foam — lidar and RF completely blocked	❌ Dead	BLIND_DRIVE

SECTION	KEY DIMENSIONS	PRIMARY CHALLENGE	FAST-LIO2 STATUS	NAVIGATION MODE
Ramp / Helix	32" wide, 4' radius, 11% grade	Tight turn on grade, IMU pitch + roll	⚠️ Degraded	SLOW_NAV
Pothole / Wide	11' × 26'	0.75" bumps, changeable boundaries	✅ Full	NORMAL_NAV

5 Zone-Aware Navigation State Machine

The robot doesn't use a single navigation strategy for the whole course — that would mean compromising everywhere. Instead, a **zone-aware state machine** monitors the robot's position against pre-defined waypoint zones and switches navigation mode appropriately. Zones are defined as simple radius checks against known waypoints — no computer vision needed.



5.1 NORMAL_NAV — Full Nav2 Smac + MPPI

This is the default mode for open sections — pothole area, hoop section, wide open areas. Nav2's Smac Hybrid-A* planner generates a global path respecting the robot's turn radius. The MPPI controller samples thousands of trajectory rollouts and picks the one that best follows the path while avoiding obstacles. The full live costmap is active.

Typical speed: **0.5–0.6 m/s**. The costmap accumulates obstacle data as the robot drives, so each subsequent run benefits from prior observations in the same session.

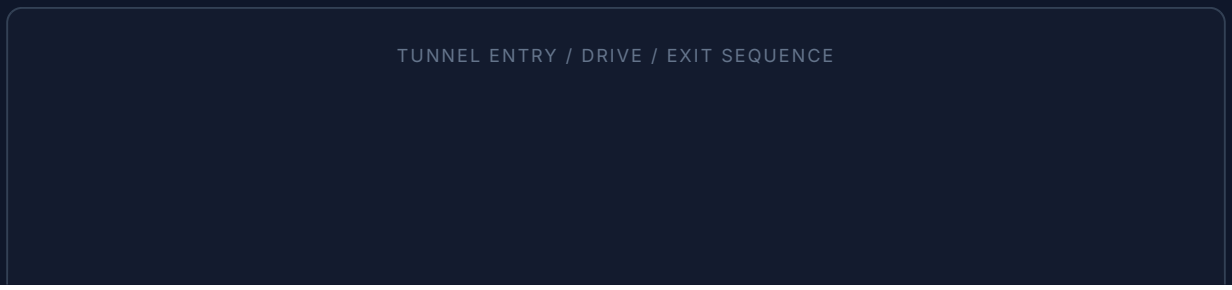
5.2 SLOW_NAV — Reduced Speed for Sensitive Sections

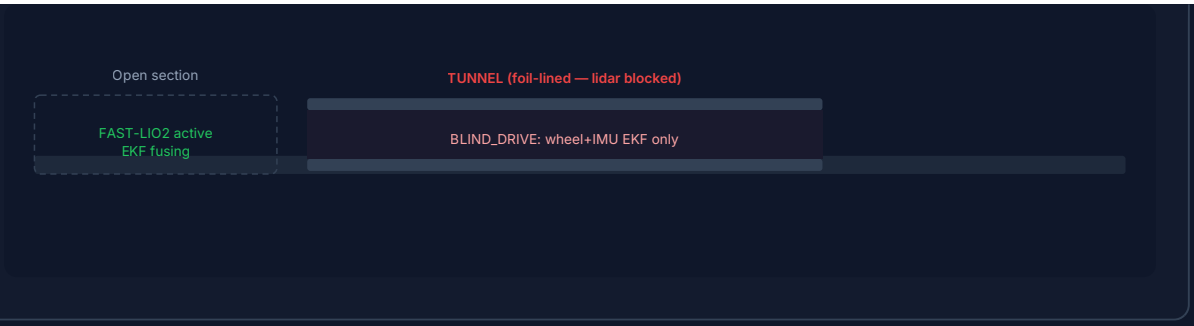
The narrow section, ramps, bank, and helix all trigger SLOW_NAV. Same Nav2 stack, but with velocity limits lowered. The key concern in the narrow section is **lateral accuracy** — with only 2 inches of clearance on each side, we need NDT-OMP localization to be solid before entering. The state machine forces an NDT re-localization check at the zone entry waypoint before proceeding.

Typical speed: **0.2–0.3 m/s**. IMU pitch threshold triggers the ramp detection automatically.

5.3 BLIND_DRIVE — Tunnel Dead Reckoning

The tunnel is the hardest section. The foil-lined foam walls block both lidar returns and RF signals. FAST-LIO2 loses its input the moment the robot enters and can't recover until it exits. This is a known, fixed-length obstacle — we handle it with deliberate dead reckoning.





Entry Sequence

- Zone waypoint triggers BLIND_DRIVE mode
- Current map→odom TF is **frozen** (cached)
- FAST-LIO2 output is ignored
- EKF2 switches to wheel+IMU only (`/wimu_odom`)
- Robot drives straight at fixed heading and low speed

Exit Sequence

- Tunnel exit zone waypoint detected (distance-based)
- Wait for FAST-LIO2 to produce a valid scan match
- NDT-OMP re-localises against `GlobalMap.pcd`
- Map→odom TF updated with corrected pose
- Return to NORMAL_NAV or SLOW_NAV

CRITICAL — IMPLEMENT FIRST

Without BLIND_DRIVE, the robot will stop at the tunnel entrance when FAST-LIO2 drops out. This is the highest-priority navigation feature to implement after the Aug 14 demo.

5.4 NAVIGATE_AROUND — Dynamic Bucket Obstacles

The obstacle section has 2–9 five-gallon buckets placed arbitrarily. These cannot be pre-mapped — they change position between runs. This is actually the section where our live-costmap approach shines brightest.

In NAVIGATE_AROUND mode, the full obstacle_layer is active. Each Hesai scan marks the buckets as occupied cells. The Smac planner finds a path through the gaps — buckets placed so that a path always exists. MPPI follows the path while continuously reacting to the costmap. If a bucket is nudged by the robot, the costmap updates within one scan cycle (100ms).

No special code is needed for this mode — it is exactly what stock Nav2 Smac + MPPI does. The key is making sure the robot enters the obstacle section with a good global pose so the initial Smac plan is in the right coordinate frame.

5.5 PUSH_THROUGH — Car Wash

The car wash has densely hanging rubber ribbons. From the lidar's perspective, these look like a solid wall across the path. If the obstacle_layer is active, Nav2 will classify the entry as blocked and refuse to proceed.

PUSH_THROUGH disables the obstacle_layer (or raises the `max_obstacle_height` threshold below the ribbon height) for the car wash zone. The robot drives straight through on a fixed heading using the pre-computed centerline waypoint. There's nothing to avoid — the ribbons are flexible and just brush past. This is the same approach as the tunnel but without the lidar blackout.

6 Nav2 Configuration for This Course

The changes needed to `nav2_params.yaml` are minimal — three lines removed to drop the static layer, and one parameter confirmed. Everything else in the existing config is already well-suited to this course.

Global Costmap — Remove static_layer

```
# BEFORE (requires .pgm):
global_costmap:
  plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
  static_layer:
    plugin: "nav2_costmap_2d::StaticLayer"
    map_topic: /map
    map_subscribe_transient_local: true

# AFTER (no .pgm needed):
global_costmap:
  plugins: ["obstacle_layer", "inflation_layer"]
  # static_layer block removed entirely
```

Key Parameters Already Correct

PARAMETER	VALUE	WHY IT MATTERS FOR THIS COURSE
`rolling_window: false`	global costmap	Costmap accumulates the whole course as robot drives — critical for long-range planning
`allow_unknown: true`	Smac planner	Robot can plan through unseen space at start — doesn't need full map before first goal
`inflation_radius: 0.45m`	global costmap	With 20" narrow section and ~28" robot width, this is tight — may need tuning to 0.3m for narrow zones
`raytrace_max_range: 6.0m`	obstacle_layer	Clears free space up to 6m ahead — keeps costmap from filling with stale marks
`motion_model: "DiffDrive"`	MPPI	Correct for our differential drive robot — constrains rollouts to physically valid motions
`vx_max: 0.6`	MPPI	Reasonable for obstacle course — increase for open sections, override per zone
`footprint: [[0.35,0.20],...]`	both costmaps	Used for collision checking. Verify this matches actual robot dimensions before competition

INFLATION RADIUS VS NARROW SECTION

The current `inflation_radius: 0.45m` on the global costmap may mark the 20" narrow section as completely impassable — the walls are only ~10" from the robot sides. When entering the narrow zone, consider temporarily reducing inflation_radius to 0.2m via a dynamic reconfigure call from the state machine.

7 Implementation Priority

Nothing described in this document is implemented yet — this is the design. Here is the recommended build order based on dependency and competition risk.

#	FEATURE	WHY THIS ORDER	TARGET
1	**Aug 14 Demo** — autonomous straight → 90° turn → straight → stop	Hard deadline. Robot executes under its own control using odometry feedback. Uses existing motion_plan_executor. Needs hardware running.	Aug 14
2	**LIO-SAM map generation** — drive course, save GlobalMap.pcd	Everything else depends on having a good map. Do this as early as possible.	Aug–Sep
3	**NDT-OMP integration** — wire into localization.launch.py	Replaces static map for localization. Required before any Nav2 competition run.	Aug–Sep
4	**Remove static_layer** from nav2_params.yaml	One-line change. Unblocks Nav2 from requiring .pgm.	Aug
5	**Dual EKF** — add ekf_wimu.yaml, update ekf_local.yaml	Smooth MPPI inputs. Needed before tuning MPPI on hardware.	Aug–Sep
6	**BLIND_DRIVE mode** — tunnel dead reckoning	Without this, robot stops at tunnel. Critical for full-course run.	Sep
7	**Zone waypoint system** — define all zone boundaries	Required to trigger mode switches. Simple radius-check against known waypoints.	Sep
8	**Car wash costmap filter** — height threshold override	Without this, Nav2 refuses to enter car wash.	Sep
9	**Inflation radius override** for narrow section	May be required to navigate the 20" narrow section with current footprint.	Sep–Oct
10	**E-stop safety review**	Required by Sep 25 — heartbeat, assertion, clearing demonstration.	Sep 25

MILESTONE CHECK

By Aug 15: hardware running, FAST-LIO2 publishing, motion_plan_demo passing. That's the foundation. Everything in this document builds on top of a working robot. Get the robot moving first.